데이터 엔지니어링 · 금융 IT 포트폴리오

# 김태형

TaeHyeong Kim

## 데이터를 수집하고 검증하는 시스템을 만들었습니다

경북대학교 컴퓨터학부 · 2027.02 졸업 예정
xogud3624@gmail.com · github.com/KTHYEONG · kthyeong.github.io

KRX-Alpha는 장중 시세 수집과 누락 구간 보충, K-Closing Alpha는 주문 전 기록으로 중복 주문 방지, 보조로 Port Entity Matching의 검증 점수와 대회 점수 차이 분석, OIBC 2026의 마감 시각 이후 정보 차단을 다룹니다.

### 프로젝트

**01 KRX-Alpha** 개인 진행 중

장중 시세 수집, 결손 탐지·보충 파이프라인

**원본 먼저 저장**
장중 파싱 없이 수신 시각만 붙여 기록

**끊김 자동 보충**
체결 번호로 빠진 구간을 찾아 채움

**수집 → 정규화 → 백업**
장 마감 후 처리 단계

**02 K-Closing Alpha** 개인

주문 전 기록으로 중복 주문을 막은 자동 집행 시스템

**주문 전 기록**
재시도에도 중복 주문 방지

**체결 내역 대조**
응답이 불확실할 때 확인

**종가 3조건 확정**
시각·장 운영 구분·가격이 모두 일치할 때

**03 Port Entity Matching** 개인 완료 보조

교차검증과 대회 점수의 차이를 원인별로 확인한 선박 기록 매칭

**0.8162 → 0.9477**
이름 변형 시험 점수 (내 컴퓨터)

**0.9595 → 0.91989**
공개 → 비공개 점수 (원인 미분석)

**04 OIBC 2026** 개인 진행 중 보조

마감 시각 이후 정보를 데이터 단계에서 막은 태양광 발전량 예측 대회 파이프라인(제출 중)

**전날 17:00 이후 차단**
예보 발행 시각이 마감을 넘으면 오류로 중단

**약 900개**
테스트 (파라미터 확장 전)

**05 SaveTax** 개인 완료 보조

중복 집계·허위 후보를 막는 공공 조달 분석(해커톤)

**45위 / 1,169명**
최종 순위 (공식 결과)

**5.93억 원**
확정 인정 금액 (접수 약 13.7억 원)

> **도움말**
> 모든 수치에 측정 방식을 표시했습니다. 실계좌 수익과 수상(Emori 팀 논문 제외)은 싣지 않았습니다. 대회 순위는 종료 후 확정된 비공개 리더보드 기준이고, 아직 결과가 나오지 않은 대회는 점수를 싣지 않았습니다. 근거는 각 저장소 README에 있습니다.

# KRX-Alpha 개인 진행 중

호가·체결 틱 수집·검증 데이터 파이프라인

증권사 실시간 API는 끊긴 구간의 체결을 다시 보내 주지 않습니다. 알파 연구 전에 틱 수집, 누락 보충, 모의 주문 검증 인프라를 먼저 만들었습니다. 알파 연구는 시작 전입니다.

기간 2026.09 ~ 진행 중

역할 수집·정규화·검증·백업·모의 주문 처리 단독 설계·구현

기술 Python asyncio/WebSocket Polars PyArrow(Parquet) SQLite zstandard Docker Compose pytest

저장소 github.com/KTHYEONG/krx-alpha

| 코드 규모 | 테스트 | 개발 기간 |
|---|---|---|
| **약 3만 줄** | **약 2,860개** | **커밋 315개** |
| src 134개 파일 · 저장소 파일 집계 | def test_ 개수 · 파라미터 확장 전 | 2026-09-08부터 · git 이력 |

## 구조




일 단위 데몬이 수집부터 백업까지 순차 실행

## 사례

CASE 01

### 장중에는 받은 그대로 저장, 해석은 장 마감 후

| 수신 | 원본 기록 | 원본 보관 | 장마감 후 | 정리본 변환 |
|---|---|---|---|---|
| 실시간 시세 메시지 | 받은 시각만 붙임 | 압축 파일에 이어 저장 | 해석·중복 제거 | 열 단위 파일 + 품질 검사 |

**P 문제** 개장 직후 시세 메시지가 한꺼번에 몰립니다. 받으면서 내용을 해석하면 처리가 밀려 일부를 놓칠 수 있습니다.

**A 분석**
- 받자마자 해석: 단순하나 해석 오류가 곧 데이터 손실이라 제외
- 원본 먼저 저장: 해석 규칙을 고쳐도 다시 계산할 수 있어 선택

**A 실행**
- 수신 단계: 받은 시각만 붙여 압축 원본 파일에 이어 저장
- 해석·중복 제거·품질 검사: 장마감 후 원본에서 정리본으로 따로 처리

**R 결과** 받는 단계와 해석 단계를 코드상 분리. 실제 데이터 손실률은 측정하지 않음.

CASE 02

### 끊긴 구간을 체결 번호로 찾아 타사 자료로 보충

| 번호 감사 | 구간 확정 | 타사 보충 | 병합 | 출처 표시 |
|---|---|---|---|---|
| 종목별 빈 구간 계산 | 시간대·종목 특정 | 키움·한국투자증권 체결 이력 | 디스크 기반 정렬 | 별도 파일로 저장 |

| | |
|---|---|
| **P** 문제 | 실시간 연결이 끊긴 동안의 체결 내역은 증권사가 다시 보내 주지 않아, 그대로 두면 영구히 빠집니다. |
| **A** 분석 | • 같은 증권사에 재요청: 지나간 체결은 받을 수 없어 제외<br>• 타사 체결 이력 조회 사용, 먼저 빈 시간대·종목을 특정해야 함<br>• 원본 보호: 보충분은 출처를 붙여 별도 저장 |
| **A** 실행 | • 감사: 체결 번호가 비는 구간을 종목별로 계산<br>• 보충: 키움증권·한국투자증권 체결 이력으로 빈 구간 채움<br>• 병합: 디스크 기반 정렬로 합치며 중복 제거 |
| **R** 결과 | 감사·보충·병합이 따로 실행되는 단계. 원본은 바꾸지 않음. 당일 체결만 제공하는 등 복구 불가 구간은 남을 수 있음. |

### 한계

- 전략(알파) 단계 없음: 신호 생성·백테스트 엔진·수익 수치 없음
- 틱 손실 0 아님: 복구 불가 구간 가능, 모의 체결 현실성 미검증

---

## K-Closing Alpha 개인

장 마감 직전 정보로 종가에 사서 다음 날 시초가에 파는 퀀트

15:20에 볼 수 있는 정보만으로 종목 점수를 매기고, 상위 3종목을 종가에 사서 다음 날 시초가에 파는 전략을 연구·자동 실행합니다. 수익률보다 숫자가 부풀려질 곳을 먼저 막았습니다. 미래 정보 차단, 비용 반영, 검증, 중복 주문 방지를 프로그램 규칙으로 고정했습니다.

기간 2026

역할 데이터 수집, 예측 모델, 검증, 주문 실행 단독 설계·구현

기술 Python LightGBM pandas PyArrow(Parquet) aiohttp pydantic pytest systemd

저장소 github.com/KTHYEONG/k-closing-alpha

| 일평균 순수익 | 샤프지수(수익 대비 변동 크기) | 비용 민감도 | 검증한 전략 수 |
|---|---|---|---|
| **+34.58bp** | **2.73** | **4칸까지 이익** | **118개** |
| 886거래일(2023-01~2026-09), 세금·호가 비용·수수료 차감 후 (1bp=0.01%) | 승률 56.3%, 최대낙폭(고점 대비 최대 하락) 28.98% | 사고팔 때 호가 간격 비용을 1~6칸으로 바꿔 시험, 6칸에서 손실 | 연구 문서의 백테스트 표 기준 |
| 백테스트 | 백테스트 | 백테스트 | 백테스트 |

### 구조



사례

CASE 01

### 주문을 먼저 기록하고 체결 내역을 확인해 중복 주문 차단

| 의도 기록 | 주문 전송 | 응답 불확실 | 체결 내역 조회 | 인정/종결 |
|---|---|---|---|---|
| 주문 내용을 디스크에 저장 | 전송 사실 기록 | 미확인 상태로 표시 | 같은 주문 탐색 | 있으면 인정 |

**P 문제** 주문 후 증권사 응답이 오지 않으면 주문이 접수됐는지 알 수 없습니다. 다시 보내면 같은 종목을 두 번 사게 됩니다.

**A 분석**
- 재시도 횟수 제한: 이미 접수된 주문의 중복은 못 막아 제외
- 전송 전 기록 + 응답이 불확실하면 증권사 체결 내역 조회: 선택

**A 실행**
- 주문 의도와 전송 사실을 디스크에 먼저 기록
- 응답이 불확실하면 '미확인' 상태로 두고, 체결 내역에 같은 주문이 있으면 그 주문으로 인정, 없을 때만 종결
- 종가 확정 판단: 시각, 장 운영 상태, 현재가와 호가 가격 일치의 3조건

**R 결과** 주문 상태 변화, 기존 주문 인정, 종가 확정 판단을 구현하고 단위 테스트 작성. 실거래 운영 기록은 없음.

한계

- 수익 수치는 모두 백테스트, 실계좌·모의투자 손익 기록 없음
- 118개 중 고른 결과라 실제 성과는 더 낮을 수 있음
- 원본 데이터가 저장소에 없어 재현 불가, 비용은 내 계좌 수수료 기준

## 보조 프로젝트

# Port Entity Matching 개인 완료

입출항 기록을 표준 선박 한 척으로 잇는 매칭 파이프라인

해양수산부 주최 DACON 항만 문제 해결 AI 대회에 개인으로 참가해, 비공개 리더보드 468명 중 106위(0.91989)로 마쳤습니다. 과제는 오타·한글 표기·빈 제원이 섞인 입출항 기록 한 줄을 약 7만 8천 척의 표준 선박 중 하나로 연결하는 것입니다. 한글 선박명이 점수의 70%라 한글 매칭에 집중했고, 발음 코드와 LLM 후보로 시작해 선박 카탈로그의 영문명을 한글로 바꿔 대조하는 역방향 채널까지 넣었습니다. 공개 점수는 0.8805에서 0.9595까지 올랐지만 비공개에서 0.04 낮아졌고, 그 원인은 분석하지 못했습니다.

기간 2026.10 ~ 2026.11

역할 대회 개인 참가, 후보 검색·순위 모델·평가 도구 단독 구현

기술 Python LightGBM RapidFuzz scikit-learn PyTorch Kanana 1.5 2.1B Midm 2.0 Mini pytest uv

저장소 github.com/KTHYEONG/port-entity-matching

| 비공개 리더보드 최종 | 공개 리더보드 점수 | 이름 변형 시험 점수 | 내부 검증 점수 |
|---|---|---|---|
| **0.91989** | **0.8805 → 0.9595** | **0.8162 → 0.9477** | **0.9910** |
| 468명 중 106위, 테스트 100% 기준 최종 순위 (대회 종료 2026.11.6) | 테스트 50% 기준, 규칙 기반 대비 최종 제출물 (한글 507/532) | 단어 순서 바꾸기·빼기 등 20가지 변형, 내 컴퓨터 모의 기준 | 선박 단위 5분할 검증, LLM 후보 추가 전 모델 |
| 운영 기록 | 운영 기록 | 백테스트 | 백테스트 |

## 사례

CASE 01

### 내부 검증 0.9875 vs 대회 점수 0.8805, 원인 분리

**P 문제** 내 컴퓨터 검증 점수와 대회 리더보드 점수 차이가 컸습니다. 원인을 모른 채 모델을 고치면 제출만 낭비합니다.

**A 분석**

- 학습 때와 제출 때 처리 방식 불일치: 예측 차이 0건, 제외
- 처음 보는 선박 0.998, 영문 톤수 오차 0.9965, 영문 오타 0.957: 제외
- 단어 순서를 바꾸거나 단어가 빠지면 정확도 0.4~0.5로 하락
- 학습 데이터는 한 선박이 늘 같은 철자, 내부 검증으로 확인 불가
- 학습 모델의 한계: 철자·순서가 달라진 이름에 약함, 한글은 LLM 후보로 보완(사례 2)

**A 실행**

- 깨끗한 내부 검증 + 이름 변형 20가지 시험 병행
- 표기 증강 학습 추가
- 단어 순서와 무관한 검색, 순서가 바뀌어도 같은 값인 특징 7종 추가

**R 결과** 이름 변형 시험 점수 0.8162에서 0.9477, 단어 순서를 바꾼 영문 이름 정확도 0.486에서 0.999. 모의 변형 기준이라 실제 테스트에서 변형이 얼마나 있는지는 알 수 없습니다. 최종 제출물도 공개 0.9595가 비공개 0.91989로 낮아졌고, 이 차이는 분석하지 못했습니다. 백테스트

**변형 시험 예시 (형식)**

| | | | |
|---|---|---|---|
| A B C | → | C A B | 단어 순서 바꿈 |
| A B C | → | A C | 단어 하나 빠짐 |
| A B C | → | A B-C | 철자·기호 표기 변화 |

**이름 변형 시험 점수 변화** 점수 백테스트




선박 단위 5분할 검증, 이름 변형 20가지 (내 컴퓨터)

## 한계

- 공개 0.9595가 비공개 0.91989로 약 0.04 낮아졌고, 원인은 분석하지 못함
- 내부 검증과 리더보드의 차이를 끝까지 설명하지 못함, 내 컴퓨터의 모의 오류로는 리더보드의 한글 오류가 재현되지 않음
- 0.9910은 LLM 후보 추가 전 값, 현재 코드의 내부 검증은 미측정
- 역방향 채널(카탈로그 이름을 미리 한글로 변환)은 규칙 허용 여부를 운영진에게 직접 확인받지 못함, 공식 답변은 학습 데이터 기반 LLM 증강까지이며 생성 코드·프롬프트는 검증 요청에 대비해 보관
- 학습 선박 제외는 대회 분할 구조를 이용한 규칙, 현장에서는 호출부호 기억이 그 역할
- 단어 하나 빠진 이름은 약함(영문 약 0.69, 한글 약 0.65, 모의 기준), 이름·톤수가 같은 자매선은 한 행만으로 구분 어려움
- 대회 데이터 재배포 금지로 저장소만으로는 학습·평가 재현 불가

# OIBC 2026 개인 진행 중

태양광 발전량 예측과 VPP 배정을 규칙 위반 없이 비교하는 파이프라인

2026 POSTECH OIBC 빅데이터 경진대회 참가 프로젝트입니다. 태양광 발전소 253곳을 두 개의 가상발전소(VPP)로 나누고, 각 VPP의 하루 전 시간별 발전량을 예측해 정산 수익을 높이는 과제입니다. 점수보다 규칙을 어기지 않는 것을 먼저 두고, 예측 마감 이후 정보 차단, 정산 기준 평가, 사전에 정한 채택 규칙을 코드로 고정했습니다. 제출 기간(11.2~11.13) 중이라 점수와 순위는 없습니다.

기간 2026.10 ~ 진행 중

역할 대회 개인 참가, 데이터 정제부터 예측·배정·제출 검증까지 단독 설계·구현

기술 Python LightGBM CatBoost scikit-learn pandas NumPy PyArrow(Parquet) SciPy Pydantic pytest uv

저장소 github.com/KTHYEONG/oibc-2026

| 코드 규모 | 테스트 | 배정 대상 |
|---|---|---|
| **약 7,800줄** | **약 900개** | **253곳 → 2개 VPP** |
| src 40개 파일 · 저장소 파일 집계 | def test_ 개수 · 파라미터 확장 전 | 한 번씩만 배정, 그룹별 용량 20,000kW 이상, 제약은 코드와 테스트로 검증 |

## 사례

CASE 01

### 예측 마감(전날 17:00) 이후 정보는 데이터 단계에서 차단

| 원천 예보 | 발행 시각 확인 | 마감 이후면 중단 | 파생 테이블 | 특성 패널 |
|---|---|---|---|---|
| 발행 시각 포함 | 대상 운영일 마감과 비교 | 오류로 멈춤 | 마감 이전 정보만 | 모델 입력 |

**P 문제** 전날 17:00 이후의 정보를 쓰면 대회 규칙 위반인데, 시계열 데이터는 실수로 미래 정보가 섞이기 쉽습니다.

**A 분석**
- 모델 쪽에서 조심하는 방식: 특성이 늘수록 빈틈이 생겨 제외
- 데이터를 파생하는 단계에서 막는 방식 채택
- 시각 표기가 섞이면 한 시간 어긋나 미래 정보가 들어갈 수 있음

**A 실행**
- 모든 시각을 KST 오른쪽 닫힘 구간으로 통일(01:00은 00~01시)
- 예보 발행 시각이 대상 운영일의 마감보다 늦으면 오류로 중단
- 마감 이후 예보를 망가뜨려도 이전 날의 특성이 바뀌지 않는지 테스트
- 입력이 규칙에 어긋나면 기본값으로 대체하지 않고 멈춤

**R 결과** 마감 게이트·시간축·정산 규칙을 단위 테스트로 고정했습니다. 게이트 모듈에만 테스트 14개가 있습니다.

## 한계

- 제출 기간(2026-11-02 ~ 11-13) 중이라 점수·순위·검증 개선 폭이 없음
- 지리 반클러스터링이 벌점을 줄이는지는 검증 결과로 확인해야 하는 가정
- 검증용 마지막 12개월은 이미 모델 비교에 쓰여 완전한 검증 데이터가 아님
- 대회 원본 데이터를 공개할 수 없어 학습·평가 재현 불가

## SaveTax 개인 완료

시중가보다 비싼 납품 물품을 찾는 감사 도구

공공조달 공개 자료에서 시중가보다 비싸게 집행된 금액을 찾는 해커톤 참가작입니다. 중복 정리, 4종 비교 근거 수집, 사람 승인 후 제출 순서로 풀었습니다. 잠정 판정에서 반려된 건에는 이의 41건을 제기했고, 인정액은 잠정 약 1.16억 원에서 최종 5.93억 원이 되어 참가자 1,169명 중 45위로 마쳤습니다.

기간 2026.10

역할 나라장터 해커톤 참가작, 단독 설계·구현

기술 Python DuckDB Parquet Decimal pytest mypy ruff

저장소 비공개 저장소

최종 순위
**45위 / 1,169명**
확정 금액이 있는 참가자 115명 중 45위, 참가자 대부분은 확정 금액 없음 (공식 결과 2026.11.9)
운영 기록

확정 인정 금액
**5.93억 원**
근거 확인과 이의 처리를 마친 최종 확정액, 접수 금액의 약 43% (공식 결과 2026.11.9)
운영 기록

잠정 인정 → 최종 확정
**1.16억 → 5.93억 원**
잠정 판정(10.30) 인정 12건·부분 인정 6건 합계에서 이의 처리 후 확정액으로, 잠정 판정에서 결과가 공개된 건은 139건 중 74건뿐이었음
운영 기록

이의 제기
**41건**
잠정 반려 31·판단 유보 4·부분 인정 6건, 신고액 합계 약 5.17억 원, 10.30에 33건 ·11.5에 8건 접수
운영 기록

제출 전 차단
**509회**
재계획 필요 405·미승인 97·근거 변경 5·계획 만료 2, 전송 전에 멈춘 횟수, 내 제출 기록 기준(고유 건수 아님)
운영 기록

테스트
**1,513개**
단위 테스트 수집 수, mypy strict·ruff 포함 통합 검사 통과를 기준으로 개발

제출 건수
**117건**
접수 139건 중 철회 22건 제외, 내 제출 기록 기준
운영 기록

접수 금액 합계
**약 13.7억 원**
제출 117건의 접수 시 금액 합, 심사 인정액 아님
운영 기록

비교 가격 근거
**4종**
타 계약·쇼핑몰·타 기관·부품 합산, 종류별 기준 배율 설정

### 사례

CASE 01

#### 반려 사유를 유형별로 나눠 이의 41건을 근거 구조로 반박

| 잠정 판정 | 유형별 묶음 | 반박문 작성 | 근거 재확인 | 이의 접수 |
|---|---|---|---|---|
| 반려 52건 사유 분류 | 랙·데스크톱·패치패널 | 근거 구조 분리 설명, 인정 사례 인용 | 가격 다시 열람 | 고유 키로 중복 방지 |

**P 문제**

마감 뒤 잠정 판정에서 공개된 74건 중 52건이 반려됐고, 52건 모두 사유에 '근거 링크가 판매 페이지가 아님'이 있었습니다. 이의 기간은 11.6까지였습니다.

**A 분석**

- 랙: 판매가 근거 옆에 적은 주최 측 공임 가산액 페이지를 판매 근거로 오인한 반려
- 데스크톱: 화면을 JS로 그리는 판매 페이지라 원본 HTML만 읽으면 가격이 비어 보임
- 같은 근거 구조의 다른 제출은 인정되어, 판정 기준 자체보다 근거 읽기의 문제로 판단
- 신고액이 큰 유형부터 묶어 처리, 건마다 따로 쓰지 않음

**A 실행**

- 반려·유보·부분 인정을 랙·데스크톱·패치패널 유형으로 나눠 이의문 초안 작성
- 판매가 근거와 가산액을 분리해 설명하고, 주최 측 공식 문서 원문과 같은 구조로 인정된 제출을 인용
- 근거 페이지는 이의 시점에 다시 열어 판매가를 재확인, 일부 건은 금액을 낮춰 조정
- 이의 요청에 고유 키를 붙여 재실행에도 중복 접수 방지, 10.30에 33건·11.5에 8건 제기

**R 결과**

이의 41건(신고액 약 5.17억 원)이 모두 접수됐고, 인정액은 잠정 약 1.16억 원에서 최종 5.93억 원이 됐습니다. 건별 이의 결과는 공개되지 않아, 잠정 때 결과가 비공개였던 65건의 영향과는 분리하지 못했습니다. 운영 기록

### 한계

- 원본 데이터와 참가자 인증 정보가 저장소에 없어 전체 실행 재현 불가, 테스트는 합성 데이터.
- 과다 집행 판정은 시중가 비교 추정, 심사에서 근거 부족일 수 있음.
- 제출 건수·접수 금액은 내 제출 기록 파일 기준(저장소 미포함)이고, 확정 금액·순위는 대회 공식 결과 페이지 기준.
- 접수 금액 약 13.7억 원 대비 확정 금액은 5.93억 원이었음.
- 먼저 등록한 사람이 물품을 가지는 대회라 속도가 중요했고, 제출 139건 중 106건을 접수 첫 이틀(10.13~14)에 냈음. 다만 판정은 마감 뒤(10.30)에 공개돼, 반려 52건 전부에 해당한 사유('근거 링크가 판매 페이지가 아님')를 접수 중에는 알 수 없었음. 근거 인용 형식을 심사 기준에 맞춰 사전 점검하는 단계를 처음부터 넣었어야 했다는 점을 배움.

## 기술 · 기타

### 사용한 기술

Python · pytest · PyArrow(Parquet) · LightGBM · pandas · pydantic · scikit-learn · uv · asyncio/WebSocket · Polars · SQLite · zstandard · Docker Compose · aiohttp · systemd · RapidFuzz · PyTorch · Kanana 1.5 2.1B

## AI 활용

AI 코딩 도구를 주로 코드를 쓰는 데 사용했고, 아이디어를 다듬거나 처음 접하는 분야를 공부할 때도 썼습니다. 수치와 동작은 직접 확인했고, 확인하지 못한 것은 성과로 쓰지 않았습니다.

| AI에 맡긴 일 | 직접 한 일 |
| --- | --- |
| 코드 작성 | 결과 수치와 동작을 직접 확인 |
| 아이디어 정리와 도메인 학습 보조 | 확인하지 못한 내용은 성과로 쓰지 않기 |

## 수치 기준 표기

**백테스트** 과거 데이터 시뮬레이션 **재현 시뮬레이션** 체결·증거금 규칙을 적용한 원장 리플레이 **평가셋 측정** 정해진 평가 데이터로 잰 값
**팀 측정** 팀 전체 측정값(제 기여분만의 값이 아님) **운영 기록** 실제로 돌린 기록

## 그 밖의 프로젝트

**auto-mobility** 실내 공간을 3D 모델로 복원하고 센서 데이터로 검증하는 온라인 현장실습 팀 프로젝트(3명). 저는 Visual SLAM 비교와 평가를 주로 맡았습니다.

**K-Stock Engine** 국내주식 다중 팩터 전략을 연구하고 가상 장부로 기록합니다. 아직 불완전한 단계라 성과는 싣지 않았습니다.

## 연락

xogud3624@gmail.com · github.com/KTHYEONG · kthyeong.github.io